Milestone 3

CSC 676 - Spring 2026

Team Alias: BayConnect

Team Members:

Nate Mugambi	Tia Parker	Sean Wang	Devin Yuan
nmugambi@sfsu.edu	**tparker@sfsu.edu**	**swang@sfsu.edu**	**dyuan3@sfsu.edu**
Github: natemugambi	**Github: tzzzp**	**Github: sw8ng**	**Github: BinaryFallow**

Repository: Link

Version History:

Version	Description	Date
v0.1	Initial draft involving a more refined version of our business rules that the group had thought of, and what they should do within our database.	2026-04-07
v0.2	Business Rules after instructor review.	2026-04-21

Table Of Contents

1. Cover Page

2. Business Rules

Business Rules

(The names next to each rule indicate who was responsible for creating and implementing business each rule)

1. Delay propagation (Sean Wang)

- Purpose: Automatically calculate how a delay at one stop affects all later stops on the same trip.
- Description: When an incident causes a delay at a specific stop, the system must insert a Delay_Propagation record for every subsequent stop on that trip, with the cumulative delay minutes.
- Challenges: The number of stops varies per trip; multiple incidents affecting the same trip must be handled.
- Assumptions: Each trip's stops are ordered by sequence_number. Delay is additive.
- Planned Approach: Use a Recursive CTE to insert propagation records.

2. Trip status change when delay exceeds threshold (Sean Wang)

- Purpose: Inform passengers when a trip becomes significantly delayed.
- Description: If Delay_Propagation for a trip has delay_minutes greater than a predefined threshold, set the trip_status to delayed.
- Challenges: The database must compare delay values with the threshold each time delays change.
- Assumptions: The threshold is predefined.
- Planned Approach: Use a trigger that updates trip status when delay records are inserted or updated.

3. Public reports stay anonymous (Sean Wang)

- Purpose: Protect user privacy in public reports.
- Description: Public reports must not show user_id or any personally identifiable information.
- Challenges: Ensure all public access uses the anonymized view rather than the base table.
- Assumptions: Admins have access to full data; public users do not.

- Planned Approach: Create a VIEW named Public_Incident_Report that excludes user_id and joins only non-sensitive columns.

4. Incident status must follow the right order (Sean Wang)

- Purpose: Ensure incident progress is tracked properly.
- Description: The only allowed status path is Reported -> In-Progress -> Resolved/Cancelled.
- Challenges: Need to know the previous status and allowed next statuses.
- Assumptions: Status names are fixed and stored in the Incident_Status lookup table.
- Planned Approach: Use a trigger or function to validate status changes.

5. Intelligent incident reporting to reduce redundancy and provide incident escalation (Tia Parker)

- Purpose: Maintain a clean incident database by preventing duplicate reports and reflecting urgency.
- Description: The database must prevent duplicate incident reports for the same location, type, and time window. Child reports must link to the existing active incident. As unique users contribute linked reports, severity updates incrementally.
- Challenges: Defining similarity thresholds for location and time. Preventing repeated reports from the same user from inflating severity.
- Assumptions: Severity follows a fixed scale (Low < Medium < High).
- Planned Approach: Use a BEFORE INSERT trigger to check for matching active incidents and update the existing row instead of inserting a duplicate.

6. Integrated maintenance and incident lifecycle (Tia Parker)

- Purpose: Ensure infrastructure repairs are actionable and statuses reflect the system state.
- Description: Infrastructure-related incidents must automatically generate linked maintenance tickets. An incident cannot be marked Resolved until all related tickets are Completed with at least one documented update.

- Challenges: Managing one-to-many relationships between incidents and maintenance tasks.
- Assumptions: Maintenance updates are stored in a child table linked to Ticket_ID.
- Planned Approach: Use AFTER INSERT triggers on Incident and BEFORE UPDATE triggers on Incident and Ticket tables.

7. Real-time system status with integrity (Tia Parker)
- Purpose: Provide passengers with an accurate overview of transit conditions.
- Description: Route and line statuses must dynamically reflect active incidents. Delay_Propagation remains the source of truth for general delays greater than zero.
- Challenges: Recalculating route status in real time without performance issues during high incident volume.
- Assumptions: Delay propagation data is the ground truth for trip timing.
- Planned Approach: Use AFTER UPDATE triggers on Incident and BEFORE UPDATE triggers on Trip.

8. Automate resolutions and escalations (Tia Parker)
- Purpose: Prevent critical issues from lingering and keep the system self-correcting.
- Description: High-severity incidents must automatically transition to Escalated or Overdue if unresolved past a threshold. Incidents must also automatically become Resolved when the last dependent maintenance task is completed.
- Challenges: Implementing time-based logic in a database environment.
- Assumptions: Escalation thresholds are stored in a configuration table.
- Planned Approach: Use an AFTER UPDATE trigger on Maintenance_Ticket and a scheduled recurring event.

9. Inactive user tracking / cleanup (Nate Mugambi)
- Purpose: Maintain system efficiency and support data collection.
- Description: Users inactive for a certain time frame are marked inactive. This can help identify active users and inactivity trends by stop or location.
- Challenges: Online inactivity may not reflect actual transit usage.

- Assumptions: Last login data is stored.
- Planned Approach: Use a scheduled recurring event to detect inactive users and perform cleanup.

10. Average rating auto-update (Nate Mugambi)
- Purpose: Keep ratings consistent.
- Description: The average rating must always reflect the aggregate of all reviews.
- Challenges: Aggregation may become slower as review counts increase.
- Assumptions: All ratings are stored in the system.
- Planned Approach: Update the average rating in real time after each review submission.

11. Real-time capacity adjustment (Nate Mugambi)
- Purpose: Improve attendance tracking accuracy.
- Description: Event capacity adjusts based on registrations and check-ins. Check-ins cannot exceed registrations, and registrations cannot exceed capacity.
- Challenges: Some users may register but never check in.
- Assumptions: Registration and check-in systems already exist.
- Planned Approach: Use triggers when users register and check in.

12. Incident report completeness (Devin Yuan)
- Purpose: Prevent incomplete reports from causing confusion for responders or operators.
- Description: Every incident report must include sufficient location information and an incident type. Incomplete reports are rejected.
- Challenges: Handling incomplete GPS data while enforcing minimum requirements.
- Assumptions: Location fields are required for routing and maintenance linking.
- Planned Approach: Use NOT NULL constraints, CHECK constraints, or a BEFORE INSERT trigger.

13. Audit logs for all severity and status changes (Devin Yuan)

- Purpose: Provide traceability for incident handling and compliance.
- Description: Changes to incident severity/status, trip status, or maintenance ticket status must create an Audit_Log record with user_id, timestamp, old_value, and new_value.
- Challenges: Performance overhead on high-volume tables.
- Assumptions: Logging is mandatory.
- Planned Approach: Use AFTER UPDATE triggers on key tables.

14. Age or account verification (Devin Yuan)
- Purpose: Reduce spam or fraudulent reports.
- Description: Users with accounts younger than X days or without email verification cannot submit high-severity incidents or restricted incident types.
- Challenges: Balancing accessibility with abuse prevention.
- Assumptions: Account creation date and verification status are stored.
- Planned Approach: Use a BEFORE INSERT trigger on Incident.

15. No overlapping maintenance window for same assets (Devin Yuan)
- Purpose: Avoid scheduling conflicts on infrastructure assets.
- Description: A new Maintenance_Ticket for an asset cannot overlap with an existing open ticket for the same asset.
- Challenges: Handling different time granularities.
- Assumptions: Assets have unique identifiers and tickets store planned start/end times.
- Planned Approach: Use a BEFORE INSERT/UPDATE trigger to detect overlapping active tickets.